

```

# =====

# AI/ML HEALTHCARE PROJECT

# Diabetes Diagnosis + Treatment Recommendation using Q-Learning

# =====


# Install if required:

# pip install numpy pandas scikit-learn imbalanced-learn matplotlib seaborn


import numpy as np
import pandas as pd
import matplotlib.pyplot as plt


from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report,
    roc_auc_score,
    roc_curve
)

from sklearn.inspection import permutation_importance


# Try importing SMOTE
try:
    from imblearn.over_sampling import SMOTE

```

```

SMOTE_AVAILABLE = True

except ImportError:

    SMOTE_AVAILABLE = False


# =====

# TASK 1: DATA PREPARATION & INDUCTIVE LEARNING

# =====


print("\n" + "=" * 70)
print("TASK 1: DATA PREPARATION & INDUCTIVE LEARNING")
print("=" * 70)


# -----
# 1. Create a sample healthcare dataset
# -----


np.random.seed(42)


n_samples = 500


age = np.random.randint(20, 81, n_samples)
blood_pressure = np.random.randint(80, 181, n_samples)
cholesterol = np.random.randint(120, 301, n_samples)
glucose = np.random.randint(70, 201, n_samples)
bmi = np.round(np.random.uniform(18, 45, n_samples), 1)
family_history = np.random.choice(
    [0, 1],
    size=n_samples,
    p=[0.65, 0.35]
)

```

```

# Create diabetes probability based on risk factors

risk_score = (
    0.035 * (age - 40)
    + 0.055 * (glucose - 100)
    + 0.12 * (bmi - 25)
    + 0.015 * (blood_pressure - 120)
    + 0.006 * (cholesterol - 200)
    + 0.9 * family_history
)

probability = 1 / (1 + np.exp(-risk_score / 4))

diabetes = (np.random.random(n_samples) < probability).astype(int)

# Make diabetic class a minority class
# This simulates the class imbalance mentioned in the problem.
if diabetes.sum() > 180:
    diabetic_indices = np.where(diabetes == 1)[0]
    np.random.seed(42)
    remove_indices = np.random.choice(
        diabetic_indices,
        size=diabetes.sum() - 180,
        replace=False
    )
    diabetes[remove_indices] = 0

# Create DataFrame
data = pd.DataFrame({
    "Age": age,
    "BloodPressure": blood_pressure,

```

```

    "Cholesterol": cholesterol,
    "Glucose": glucose,
    "BMI": bmi,
    "FamilyHistory": family_history,
    "Diabetes": diabetes
})

print("\nFirst 10 records:")
print(data.head(10))

print("\nDataset shape:")
print(data.shape)

print("\nClass distribution:")
print(data["Diabetes"].value_counts())

print("\nClass distribution percentage:")
print(data["Diabetes"].value_counts(normalize=True) * 100)

# -----
# 2. Check missing values
# -----

print("\nMissing values:")
print(data.isnull().sum())

# -----
# 3. Features and target
# -----

```

```

X = data.drop("Diabetes", axis=1)
y = data["Diabetes"]

print("\nTarget variable: Diabetes")
print("0 = Non-Diabetic")
print("1 = Diabetic")

print("\nFeatures:")
print(list(X.columns))

# -----
# 4. Train-Test Split 70:30
# -----

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.30,
    random_state=42,
    stratify=y
)

print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))

# -----
# 5. Feature Scaling
# -----

```

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

```
# -----
```

```
# 6. Handle Class Imbalance using SMOTE
```

```
# -----
```

```
print("\nClass distribution BEFORE SMOTE:")
```

```
print(pd.Series(y_train).value_counts())
```

```
if SMOTE_AVAILABLE:
```

```
    smote = SMOTE(random_state=42)
```

```
    X_train_balanced, y_train_balanced = smote.fit_resample(
        X_train_scaled,
        y_train
    )
```

```
    print("\nSMOTE applied successfully.")
```

```
else:
```

```
    # Fallback if imbalanced-learn is not installed.
```

```
    # Class weights are used later in the models.
```

```
    X_train_balanced = X_train_scaled
```

```
    y_train_balanced = y_train
```

```
print("\nSMOTE library not available.")
print("Using class-weight balancing instead.")
```

```
print("\nClass distribution AFTER balancing:")
print(pd.Series(y_train_balanced).value_counts())
```

```
# =====
# TASK 2: DECISION TREE FOR DIAGNOSIS
# =====
```

```
print("\n" + "=" * 70)
print("TASK 2: DECISION TREE FOR DIAGNOSIS")
print("=" * 70)
```

```
# -----
# 1. Train Decision Tree
# -----
```

```
decision_tree = DecisionTreeClassifier(
    criterion="gini",
    max_depth=5,
    class_weight="balanced",
    random_state=42
)
```

```
decision_tree.fit(
    X_train_balanced,
    y_train_balanced
```

```
)
```

```
# -----
```

```
# 2. Predictions
```

```
# -----
```

```
y_pred_tree = decision_tree.predict(X_test_scaled)
```

```
y_prob_tree = decision_tree.predict_proba(X_test_scaled)[:, 1]
```

```
# -----
```

```
# 3. Evaluation function
```

```
# -----
```

```
def evaluate_model(model_name, y_true, y_pred, y_prob):
```

```
    accuracy = accuracy_score(y_true, y_pred)
```

```
    precision = precision_score(
```

```
        y_true,
```

```
        y_pred,
```

```
        zero_division=0
```

```
    )
```

```
    recall = recall_score(
```

```
        y_true,
```

```
        y_pred,
```

```
        zero_division=0
```

```
    )
```

```

f1 = f1_score(
    y_true,
    y_pred,
    zero_division=0
)

auc = roc_auc_score(y_true, y_prob)

cm = confusion_matrix(y_true, y_pred)

tn, fp, fn, tp = cm.ravel()

print("\n-----")
print(model_name)
print("-----")

print("Accuracy :", round(accuracy, 4))
print("Precision :", round(precision, 4))
print("Recall   :", round(recall, 4))
print("F1-Score :", round(f1, 4))
print("AUC-ROC  :", round(auc, 4))

print("\nConfusion Matrix:")
print(cm)

print("\nTrue Negatives :", tn)
print("False Positives:", fp)
print("False Negatives:", fn)
print("True Positives :", tp)

print("\nClassification Report:")

```

```
print(
    classification_report(
        y_true,
        y_pred,
        target_names=["Non-Diabetic", "Diabetic"],
        zero_division=0
    )
)
```

```
return {
    "Accuracy": accuracy,
    "Precision": precision,
    "Recall": recall,
    "F1-Score": f1,
    "AUC-ROC": auc,
    "False Negatives": fn
}
```

```
tree_results = evaluate_model(
    "DECISION TREE",
    y_test,
    y_pred_tree,
    y_prob_tree
)
```

```
# -----
```

```
# 4. Display first three levels of Decision Tree
```

```
# -----
```

```
print("\nDecision Tree structure:")
print("Maximum depth:", decision_tree.get_depth())
print("Number of leaves:", decision_tree.get_n_leaves())
```

```
plt.figure(figsize=(18, 10))
```

```
plot_tree(
    decision_tree,
    feature_names=X.columns,
    class_names=["Non-Diabetic", "Diabetic"],
    filled=True,
    max_depth=3,
    rounded=True
)
```

```
plt.title("First Three Levels of Decision Tree")
plt.tight_layout()
plt.show()
```

```
# -----
```

```
# 5. Decision Tree Feature Importance
```

```
# -----
```

```
tree_importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": decision_tree.feature_importances_
}).sort_values(
    by="Importance",
    ascending=False
)
```

```
print("\nDecision Tree Feature Importance:")
```

```
print(tree_importance)
```

```
print("\nTop 3 Decision Tree Predictors:")
```

```
print(tree_importance.head(3))
```

```
# -----
```

```
# 6. Gini-based root feature
```

```
# -----
```

```
root_feature_index = decision_tree.tree_.feature[0]
```

```
if root_feature_index >= 0:
```

```
    root_feature = X.columns[root_feature_index]
```

```
    print("\nDecision Tree Root Feature:", root_feature)
```

```
# =====
```

```
# TASK 2(c): POST-PRUNING / COST-COMPLEXITY PRUNING
```

```
# =====
```

```
print("\n" + "=" * 70)
```

```
print("TASK 2(c): POST-PRUNING")
```

```
print("=" * 70)
```

```
# -----
```

```
# Find cost-complexity pruning path
```

```
# -----
```

```

unpruned_tree = DecisionTreeClassifier(
    criterion="gini",
    class_weight="balanced",
    random_state=42
)

unpruned_tree.fit(
    X_train_balanced,
    y_train_balanced
)

path = unpruned_tree.cost_complexity_pruning_path(
    X_train_balanced,
    y_train_balanced
)

ccp_alphas = path.ccp_alphas

# -----
# Select a pruning parameter
# -----

# Ignore the last alpha which normally produces a tree
# containing only the root.
if len(ccp_alphas) > 2:
    selected_alpha = ccp_alphas[len(ccp_alphas) // 2]
else:
    selected_alpha = ccp_alphas[-1]

```

```

pruned_tree = DecisionTreeClassifier(
    criterion="gini",
    class_weight="balanced",
    random_state=42,
    ccp_alpha=selected_alpha
)

pruned_tree.fit(
    X_train_balanced,
    y_train_balanced
)

y_pred_pruned = pruned_tree.predict(X_test_scaled)
y_prob_pruned = pruned_tree.predict_proba(X_test_scaled)[:, 1]

pruned_results = evaluate_model(
    "POST-PRUNED DECISION TREE",
    y_test,
    y_pred_pruned,
    y_prob_pruned
)

print("\nSelected ccp_alpha:", selected_alpha)
print("Pruned Tree Depth:", pruned_tree.get_depth())
print("Pruned Tree Leaves:", pruned_tree.get_n_leaves())

# -----
# Compare pre-pruned and post-pruned
# -----

```

```
print("\nPre-Pruning vs Post-Pruning")
print("-----")
```

```
comparison_tree = pd.DataFrame({
    "Metric": [
        "Accuracy",
        "Precision",
        "Recall",
        "F1-Score",
        "AUC-ROC",
        "False Negatives"
    ],
    "Pre-Pruned Tree": [
        tree_results["Accuracy"],
        tree_results["Precision"],
        tree_results["Recall"],
        tree_results["F1-Score"],
        tree_results["AUC-ROC"],
        tree_results["False Negatives"]
    ],
    "Post-Pruned Tree": [
        pruned_results["Accuracy"],
        pruned_results["Precision"],
        pruned_results["Recall"],
        pruned_results["F1-Score"],
        pruned_results["AUC-ROC"],
        pruned_results["False Negatives"]
    ]
})
```

```
print(comparison_tree.round(4))
```

```
# =====

# TASK 3: STATISTICAL LEARNING

# =====

print("\n" + "=" * 70)
print("TASK 3: STATISTICAL LEARNING - LOGISTIC REGRESSION")
print("=" * 70)


# -----
# 1. Train Logistic Regression
# -----

logistic_model = LogisticRegression(
    class_weight="balanced",
    max_iter=2000,
    random_state=42
)

logistic_model.fit(
    X_train_balanced,
    y_train_balanced
)


# -----
# 2. Predictions
# -----
```

```
y_pred_logistic = logistic_model.predict(X_test_scaled)
y_prob_logistic = logistic_model.predict_proba(X_test_scaled)[:, 1]
```

```
# -----
```

```
# 3. Evaluate Logistic Regression
```

```
# -----
```

```
logistic_results = evaluate_model(
    "LOGISTIC REGRESSION",
    y_test,
    y_pred_logistic,
    y_prob_logistic
)
```

```
# -----
```

```
# 4. Compare Decision Tree and Logistic Regression
```

```
# -----
```

```
print("\n" + "=" * 70)
print("MODEL PERFORMANCE COMPARISON")
print("=" * 70)
```

```
comparison = pd.DataFrame({
    "Metric": [
        "Accuracy",
        "Precision",
        "Recall",
        "F1-Score",
        "AUC-ROC",
```

```

        "False Negatives"
    ],
    "Decision Tree": [
        tree_results["Accuracy"],
        tree_results["Precision"],
        tree_results["Recall"],
        tree_results["F1-Score"],
        tree_results["AUC-ROC"],
        tree_results["False Negatives"]
    ],
    "Logistic Regression": [
        logistic_results["Accuracy"],
        logistic_results["Precision"],
        logistic_results["Recall"],
        logistic_results["F1-Score"],
        logistic_results["AUC-ROC"],
        logistic_results["False Negatives"]
    ]
})

```

```

print(comparison.round(4))

```

```

# =====
# TASK 3(b): FEATURE IMPORTANCE
# =====

```

```

print("\n" + "=" * 70)
print("FEATURE IMPORTANCE ANALYSIS")
print("=" * 70)

```

```

# -----
# Logistic Regression coefficients
# -----

logistic_importance = pd.DataFrame({
    "Feature": X.columns,
    "Coefficient": logistic_model.coef_[0]
})

logistic_importance["Absolute_Importance"] = (
    logistic_importance["Coefficient"].abs()
)

logistic_importance = logistic_importance.sort_values(
    by="Absolute_Importance",
    ascending=False
)

print("\nLogistic Regression Feature Importance:")
print(logistic_importance)

print("\nTop 3 Logistic Regression Predictors:")
print(
    logistic_importance[
        ["Feature", "Coefficient"]
    ].head(3)
)

```

```

# -----
# Compare top 3 predictors
# -----

top3_tree = set(
    tree_importance.head(3)["Feature"]
)

top3_logistic = set(
    logistic_importance.head(3)["Feature"]
)

common_features = top3_tree.intersection(
    top3_logistic
)

print("\nTop 3 Decision Tree Predictors:")
print(list(top3_tree))

print("\nTop 3 Logistic Regression Predictors:")
print(list(top3_logistic))

print("\nCommon Top Predictors:")
print(list(common_features))

if len(common_features) >= 2:
    print(
        "\nResult: Both models show strong agreement "
        "among the important predictors."
    )
else:

```

```

print(
    "\nResult: The models show limited agreement. "
    "This may occur because the models capture relationships differently."
)

# =====
# ROC CURVE COMPARISON
# =====

print("\n" + "=" * 70)
print("ROC-AUC COMPARISON")
print("=" * 70)

fpr_tree, tpr_tree, _ = roc_curve(
    y_test,
    y_prob_tree
)

fpr_logistic, tpr_logistic, _ = roc_curve(
    y_test,
    y_prob_logistic
)

plt.figure(figsize=(8, 6))

plt.plot(
    fpr_tree,
    tpr_tree,
    label=f"Decision Tree AUC = {tree_results['AUC-ROC']:.3f}"
)

```

```
plt.plot(
    fpr_logistic,
    tpr_logistic,
    label=f"Logistic Regression AUC = {logistic_results['AUC-ROC']:.3f}"
)
```

```
plt.plot(
    [0, 1],
    [0, 1],
    linestyle="--",
    label="Random Classifier"
)
```

```
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve Comparison")
plt.legend()
plt.grid(True)
plt.show()
```

```
# =====
# TASK 4: REINFORCEMENT LEARNING
# =====
```

```
print("\n" + "=" * 70)
print("TASK 4: REINFORCEMENT LEARNING FOR TREATMENT")
print("=" * 70)
```

```
# -----  
# 1. Define MDP  
# -----
```

```
states = [  
    "Low Risk",  
    "Moderate Risk",  
    "High Risk",  
    "Controlled"  
]
```

```
actions = [  
    "Diet",  
    "Exercise",  
    "Medication",  
    "Monitor"  
]
```

```
print("\nStates:")  
for i, state in enumerate(states):  
    print(i, "=", state)
```

```
print("\nActions:")  
for i, action in enumerate(actions):  
    print(i, "=", action)
```

```
# -----  
# 2. Reward Function  
# -----
```

Illustrative rewards.

In real healthcare, rewards must be based on validated

clinical outcomes.

reward_table = {

"Low Risk": {

"Diet": 5,

"Exercise": 4,

"Medication": -2,

"Monitor": 3

},

"Moderate Risk": {

"Diet": 6,

"Exercise": 7,

"Medication": 3,

"Monitor": 2

},

"High Risk": {

"Diet": 4,

"Exercise": 5,

"Medication": 9,

"Monitor": 3

},

"Controlled": {

"Diet": 3,

"Exercise": 4,

"Medication": 1,

```
    "Monitor": 8
  }
}
```

```
# -----
# 3. Transition Dynamics
# -----
```

```
# Each state-action pair gives the next health state.
# This is a simplified simulation for the practical.
```

```
transition = {
```

```
  "Low Risk": {
    "Diet": "Controlled",
    "Exercise": "Controlled",
    "Medication": "Low Risk",
    "Monitor": "Low Risk"
  },
```

```
  "Moderate Risk": {
    "Diet": "Low Risk",
    "Exercise": "Low Risk",
    "Medication": "Controlled",
    "Monitor": "Moderate Risk"
  },
```

```
  "High Risk": {
    "Diet": "Moderate Risk",
    "Exercise": "Moderate Risk",
```

```

        "Medication": "Controlled",
        "Monitor": "High Risk"
    },

    "Controlled": {
        "Diet": "Controlled",
        "Exercise": "Controlled",
        "Medication": "Moderate Risk",
        "Monitor": "Controlled"
    }
}

# =====
# Q-LEARNING PARAMETERS
# =====

alpha = 0.5
gamma = 0.9
epsilon = 0.2

num_episodes = 10

q_table = pd.DataFrame(
    0.0,
    index=states,
    columns=actions
)

print("\nInitial Q-Table:")
print(q_table)

```

```

# -----
# Q-Learning Algorithm
# -----

np.random.seed(42)

q_history = []

for episode in range(1, num_episodes + 1):

    current_state = np.random.choice(states)

    for step in range(10):

        # Epsilon-greedy action selection

        if np.random.random() < epsilon:

            action = np.random.choice(actions)

        else:

            action = q_table.loc[
                current_state
            ].idxmax()

        # Obtain reward
        reward = reward_table[
            current_state

```

```
][action]
```

```
# Find next state
```

```
next_state = transition[
```

```
    current_state
```

```
][action]
```

```
# Current Q value
```

```
old_q = q_table.loc[
```

```
    current_state,
```

```
    action
```

```
]
```

```
# Maximum future Q value
```

```
max_future_q = q_table.loc[
```

```
    next_state
```

```
].max()
```

```
# Q-Learning update
```

```
new_q = old_q + alpha * (
```

```
    reward
```

```
    + gamma * max_future_q
```

```
    - old_q
```

```
)
```

```
q_table.loc[
```

```
    current_state,
```

```
    action
```

```
] = new_q
```

```
# Save history
```

```
q_history.append({
    "Episode": episode,
    "State": current_state,
    "Action": action,
    "Reward": reward,
    "Old Q": old_q,
    "New Q": new_q
})
```

```
# Move to next state
current_state = next_state
```

```
# Stop if controlled
if current_state == "Controlled":
    break
```

```
# -----
```

```
# Show Q-learning updates
```

```
# -----
```

```
q_history_df = pd.DataFrame(q_history)
```

```
print("\nQ-Learning Update History:")
```

```
print(q_history_df.head(20).round(4))
```

```
# -----
```

```
# Show at least 3 state-action pairs
```

```
# -----
```

```

print("\nSample Q-Table Updates:")
print(
    q_history_df[
        ["Episode", "State", "Action", "Reward", "Old Q", "New Q"]
    ].head(10).round(4)
)

```

```

# -----
# Final Q-Table
# -----

```

```

print("\n" + "=" * 70)
print("FINAL Q-TABLE")
print("=" * 70)

```

```

print(q_table.round(4))

```

```

# =====
# FINAL LEARNED POLICY
# =====

```

```

print("\n" + "=" * 70)
print("FINAL LEARNED POLICY")
print("=" * 70)

```

```

policy = {}

```

```

for state in states:

```

```
best_action = q_table.loc[
    state
].idxmax()
```

```
best_value = q_table.loc[
    state,
    best_action
]
```

```
policy[state] = best_action
```

```
print(
    f"{state:15s} -> {best_action:12s}"
    f" Q-value = {best_value:.4f}"
)
```

```
# =====
```

```
# CONVERGENCE CHECK
```

```
# =====
```

```
print("\n" + "=" * 70)
```

```
print("CONVERGENCE ANALYSIS")
```

```
print("=" * 70)
```

```
# Run a second training comparison from a copy
```

```
old_q_table = q_table.copy()
```

```
# Additional training
```

```
for episode in range(11, 21):
```

```
current_state = np.random.choice(states)
```

```
for step in range(10):
```

```
    if np.random.random() < epsilon:
```

```
        action = np.random.choice(actions)
```

```
    else:
```

```
        action = q_table.loc[
```

```
            current_state
```

```
        ].idxmax()
```

```
    reward = reward_table[
```

```
        current_state
```

```
    ][action]
```

```
    next_state = transition[
```

```
        current_state
```

```
    ][action]
```

```
    old_q = q_table.loc[
```

```
        current_state,
```

```
        action
```

```
    ]
```

```
    max_future_q = q_table.loc[
```

```
        next_state
```

```
    ].max()
```

```
    new_q = old_q + alpha * (
```

```
        reward
```

```
        + gamma * max_future_q
```

```

        - old_q
    )

    q_table.loc[
        current_state,
        action
    ] = new_q

    current_state = next_state

    if current_state == "Controlled":
        break

max_change = np.max(
    np.abs(
        q_table.values -
        old_q_table.values
    )
)

epsilon_convergence = 0.001

print("Maximum Q-value change:", round(max_change, 6))
print("Convergence threshold:", epsilon_convergence)

if max_change < epsilon_convergence:
    print("Status: Q-Learning has converged.")
else:
    print("Status: More training episodes are required.")

```

```
# =====
```

```
# FINAL REPORT SUMMARY
```

```
# =====
```

```
print("\n" + "=" * 70)
```

```
print("PROJECT SUMMARY")
```

```
print("=" * 70)
```

```
print("""
```

1. Dataset prepared with healthcare features.
2. Target variable: Diabetes.
3. Dataset split into 70% training and 30% testing.
4. Class imbalance handled using SMOTE/class weighting.
5. Decision Tree trained for diabetes diagnosis.
6. Accuracy, Precision, Recall, F1-score and False Negatives calculated.
7. Decision Tree feature importance calculated.
8. Post-pruning applied using cost-complexity pruning.
9. Logistic Regression trained as a statistical learning model.
10. Decision Tree and Logistic Regression compared using AUC-ROC.
11. Top diabetes predictors identified from both models.
12. Treatment recommendation formulated as an MDP.
13. Q-Learning applied to treatment recommendation.
14. Final Q-table and learned policy generated.
15. Convergence checked using Q-value changes.

```
IMPORTANT:
```

The treatment recommendations and Q-Learning environment in this program are educational simulations. They must not be used to make actual medical treatment or medication decisions.

```
""")
```

```
print("\nProgram execution completed successfully.
```